

Why Data Visualization?

- Helps communicate insights from raw data.
- Converts tables & numbers into patterns, trends, and anomalies.
- Makes decision-making faster and easier.

Introduction

- **Matplotlib**: Low-level, flexible, customizable plotting library in Python.
- **Seaborn**: High-level statistical plotting library built on Matplotlib, easier for beautiful visualizations. Built on top of **Matplotlib**.
- Use case difference:
 - **Matplotlib** → Control & customization
 - **Seaborn** → Quick, attractive statistical graphics

Feature	Matplotlib	Seaborn
Level	Low-level	High-level
Customization	Full control	Limited (but neat defaults)
Statistical plots	Manual	Built-in
Default look	Basic	Attractive

Matplotlib references

1. Figure & Axes Management

- `plt.figure(figsize=(w,h), dpi=100, facecolor="w", edgecolor="k")`
Creates a new figure.

- figsize: tuple, width × height in inches
- dpi: resolution
- facecolor: background color
- edgecolor: border color
- **plt.subplots(nrows=1, ncols=1, figsize=(w,h), sharex=False, sharey=False)**
Creates subplots grid. Returns (fig, ax) objects.
 - nrows, ncols: grid layout
 - sharex, sharey: share axis across subplots
- **plt.subplot(nrows, ncols, index)**
Adds a subplot at given position.

2. Line & Scatter Plots

- **plt.plot(x, y, color=None, linestyle=None, linewidth=None, marker=None, label=None)**
 - color: 'r','g','b' or hex codes ("ff0000")
 - linestyle: '-', '--', '-.', ':'
 - linewidth: thickness
 - marker: 'o','s','^','*','D','x'
 - label: for legend
- **plt.scatter(x, y, s=None, c=None, marker=None, cmap=None, alpha=None, label=None)**
 - s: size of points
 - c: colors or colormap values
 - marker: shape of marker
 - cmap: colormap ('viridis','plasma','coolwarm')
 - alpha: transparency (0–1)

3. Bar & Pie Charts

- **plt.bar(x, height, width=0.8, color=None, edgecolor=None, linewidth=None, label=None)**
 - x: categories
 - height: bar heights
 - width: bar width
 - color: fill color
 - edgecolor: outline color
 - linewidth: border thickness

- **plt.barh(y, width, height=0.8, color=None, align="center")**
Horizontal bar plot.
- **plt.pie(x, labels=None, colors=None, autopct=None, startangle=0, explode=None, shadow=False)**
 - x: values
 - labels: category names
 - colors: list of colors
 - autopct: format string ('%1.1f%%')
 - startangle: rotation
 - explode: offset slices
 - shadow: 3D effect

4. Histograms

- **plt.hist(x, bins=10, range=None, density=False, cumulative=False, color=None, edgecolor=None)**
 - bins: number of bins or sequence
 - density: normalize to probability
 - cumulative: cumulative histogram

5. Axis Control

- **plt.xlim(left=None, right=None), plt.ylim(bottom=None, top=None)**
 - Set axis limits.
- **plt.xticks(ticks=None, labels=None, rotation=None)**
- **plt.yticks(ticks=None, labels=None, rotation=None)**
 - Customize tick positions & labels.
- **plt.tick_params(axis="both", which="major/minor", direction="in/out", length=5, width=1, colors=None)**
 - Full control of tick styles.

6. Labels, Titles, Legends

- **plt.xlabel("label", fontsize=None, color=None, loc=None)**
- **plt.ylabel("label", fontsize=None, color=None, loc=None)**
- **plt.title("Title", fontsize=None, color=None, loc="center")**
- **plt.legend(loc="best", fontsize=None, title=None, shadow=False, ncol=1)**

- loc: 'best', 'upper left', 'lower right', 'center', 'outside'
- ncol: number of columns
- **plt.text(x, y, "string", fontsize=None, color=None, rotation=None)**
- **plt.annotate("text", xy=(x,y), xytext=(x2,y2), arrowprops=dict())**

7. Grid & Styles

- **plt.grid(True/False, which="major/minor", axis="x/y/both", linestyle="--", linewidth=0.5, color="gray")**
- **plt.style.use("ggplot")** (preset themes: "seaborn", "classic", "fivethirtyeight", "dark_background")

8. Saving Figures

- **plt.savefig("filename.png", dpi=300, bbox_inches="tight", facecolor=None, transparent=False)**
 - dpi: resolution
 - bbox_inches="tight" trims whitespace
 - transparent=True saves with transparent background

9. Other Useful Functions

- **plt.fill_between(x, y1, y2=0, where=None, interpolate=False, alpha=None)**
Fill area under curves.
- **plt.stackplot(x, y1, y2, labels=[])**
Stacked area plots.
- **plt.errorbar(x, y, yerr=None, xerr=None, fmt='o', ecolor="black", capsize=5)**
Plot with error bars.
- **plt.boxplot(data, vert=True, patch_artist=True, notch=False)**
Box-and-whisker plots.
- **plt.violinplot(dataset, showmeans=False, showmedians=True)**
Advanced distribution plot.

Seaborn Functions & Attributes Reference

Seaborn is built on top of Matplotlib. You can call **Seaborn functions** and also customize plots with **Matplotlib's plt or ax methods**.

1. Setup & Styling

- **sns.set_style(style)**
 - Styles: "white", "dark", "whitegrid", "darkgrid", "ticks".

- **sns.set_context(context)**
 - Options: "paper", "notebook", "talk", "poster".
 - Changes scale of fonts/labels.
 - **sns.set_palette(palette)**
 - Palettes: "deep", "muted", "bright", "pastel", "dark", "colorblind".
 - Or pass custom list of colors.
-

2. Distribution Plots

sns.histplot(data, x=None, y=None, bins=None, kde=False, hue=None, multiple="layer", stat="count", element="bars")

- x / y: numeric variable
 - bins: number of bins
 - kde: add Kernel Density Estimation curve
 - hue: group by category
 - multiple: "layer", "stack", "dodge"
 - stat: "count", "probability", "density"
 - element: "bars", "step", "poly"
-

◆ **sns.kdeplot(data, x=None, y=None, hue=None, fill=False, bw_adjust=1)**

- fill: fill under curve
 - bw_adjust: bandwidth of smoothing
-

◆ **sns.displot(data, x=None, kind="hist"/"kde"/"ecdf", col=None, row=None, hue=None)**

- Combines histogram, KDE, ECDF with faceting.
-

3. Categorical Plots

◆ **sns.barplot(data, x=None, y=None, hue=None, estimator=mean, ci=95, palette=None, dodge=True)**

- estimator: function (e.g., np.mean, np.median, len)
 - ci: confidence interval (95, or None)
 - dodge: separate bars per category
-

◆ **sns.countplot(data, x=None, y=None, hue=None, palette=None)**

- Shows frequency counts.
-

◆ **sns.boxplot(data, x=None, y=None, hue=None, orient="v", palette=None, showmeans=False)**

- orient: orientation "h" or "v"
 - showmeans: mark mean value
-

◆ **sns.violinplot(data, x=None, y=None, hue=None, split=False, inner="box"/"quartile"/"point")**

- split=True: split violin for hue categories
- inner: representation inside violin

-
- ◆ **sns.stripplot(data, x=None, y=None, hue=None, jitter=True, dodge=False)**
 - Adds scatter points (for small datasets).
-

- ◆ **sns.swarmplot(data, x=None, y=None, hue=None, dodge=False)**
 - Like stripplot but avoids overlap.
-

4. Relational Plots

- ◆ **sns.scatterplot(data, x=None, y=None, hue=None, style=None, size=None, palette=None, markers=True)**
 - hue: color groups
 - style: marker style groups
 - size: point sizes
-
- ◆ **sns.lineplot(data, x=None, y=None, hue=None, style=None, markers=False, dashes=True, ci=95, estimator=mean)**
 - ci: confidence interval shading
 - estimator: function applied (mean, median)
-
- ◆ **sns.relplot(data, x=None, y=None, hue=None, kind="scatter"/"line", col=None, row=None)**
 - High-level relational plotting with facets.
-

5. Matrix / Heatmap Plots

- ◆ **sns.heatmap(data, vmin=None, vmax=None, cmap="viridis", center=None, annot=False, fmt=".2f", linewidths=0, linecolor="white")**
 - annot=True: show values inside cells
 - fmt: formatting (".2f", "d")
 - cmap: colormap ("coolwarm", "Blues", "magma")
-
- ◆ **sns.clustermap(data, cmap="viridis", method="average", metric="euclidean", annot=False, figsize=(10,10))**
 - Clustering rows & columns with dendrograms.
-

6. Regression & Statistical Plots

- ◆ **sns.regplot(data, x=None, y=None, scatter=True, fit_reg=True, ci=95, order=1)**
 - Fits regression line (linear or polynomial with order).
-
- ◆ **sns.lmplot(data, x=None, y=None, hue=None, col=None, row=None, order=1, ci=95)**
 - Higher-level wrapper for regression + faceting.
-

7. Multivariate / Pairwise Plots

- ◆ `sns.pairplot(data, hue=None, kind="scatter"/"kde"/"reg", diag_kind="hist"/"kde")`
 - Pairwise plots for all numeric variables.
-
- ◆ `sns.jointplot(data, x=None, y=None, kind="scatter"/"kde"/"hex"/"reg")`
 - Joint distribution of two variables with marginal histograms.
-

8. Facet Grids

- ◆ `sns.FacetGrid(data, col=None, row=None, hue=None)`
- Create grid of subplots based on categories.

```
g = sns.FacetGrid(tips, col="sex", row="time", hue="smoker")
g.map(sns.scatterplot, "total_bill", "tip")
g.add_legend()
```

9. Themes & Palettes

- `sns.color_palette("pastel")` → returns color list.
- `sns.set_theme(style="whitegrid", palette="muted", context="notebook")`

10. Saving & Integration

Since Seaborn returns a Matplotlib Axes object, you can still use:

- `ax.set_title("Title")`
- `ax.set_xlabel("Label")`
- `ax.set_ylabel("Label")`
- `plt.savefig("plot.png", dpi=300)`